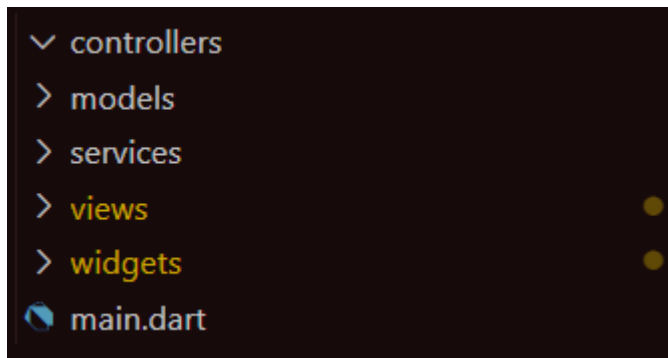


TUGAS FLUTTER API-1: MEMBUAT REGISTER PELANGGAN TOKO ONLINE

```
# versions available, run `flutter pub outdated`.
dependencies:
  flutter:
    sdk: flutter

  # The following adds the Cupertino Icons font to your application.
  # Use with the CupertinoIcons class for iOS style icons.
  cupertino_icons: ^1.0.8
  http: ^1.2.2
  shared_preferences: ^2.3.3
```

Tambahkan Http di pubspec.yaml atau gunakan flutter pub add http



Lalu buatlah folder seperti ini dan buat file register user di dalam folder views

```
import 'package:toko_online/views/register_user_view.dart';

import 'package:toko_online/views/register_user_view.dart';

Run | Debug | Profile
void main() {
  runApp(MaterialApp(
    initialRoute: '/',
    routes: {
      '/': (context) => RegisterUserView(),
    },
  ));
}
```

Setelah itu buat halaman main.dart seperti itu untuk menampilkan halaman register

```
models > response_data_map.dart > ...
class ResponseDataMap {
  bool status;
  String message;
  Map? data;
  ResponseDataMap({required this.status, required this.message, this.data});
}
```

- data_map.dart, untuk menyimpan status, pesan, dan map data

```
models > response_data_list.dart > ...
class ResponseDataList {
  bool status;
  String message;
  List? data;
  ResponseDataList({required this.status, required this.message, this.data});
}
```

- data_list.dart, untuk menyimpan status, pesan, dan list data

Setelah itu buat data_map dan data_list di models dengan script seperti di atas, keduanya digunakan untuk menangani respons API

```
class AlertMessage {
  showAlert(BuildContext context, message, status) {
    Color? warnafill;
    Color warnagaris;
    if (status) {
      warnafill = Colors.green[200];
      warnagaris = Colors.green;
    } else {
      warnafill = Colors.pink[200];
      warnagaris = Colors.red;
    }

    SnackBar snackBar = SnackBar(
      backgroundColor: Colors.transparent,
      elevation: 0,
      content: Container(
        padding: EdgeInsets.all(8),
        decoration: BoxDecoration(
          color: warnafill,
          border: Border.all(color: warnagaris, width: 3),
          boxShadow: [
            BoxShadow(
              color: Color(0x19000000),
              spreadRadius: 2.0,
              blurRadius: 8.0,
              offset: Offset(2, 4),
            ) // BoxShadow
          ],
          borderRadius: BorderRadius.circular(4),
        ), // BoxDecoration
      child: Row(
        children: [
          Icon(Icons.favorite, color: Color.fromARGB(255, 24, 60, 121)),
```

```

        Padding(
          padding: EdgeInsets.only(left: 8.0),
          child: Text(message,
            style:
              TextStyle(color: ☐const Color.fromARGB(255, 0, 0, 0))), // Text
        ), // Padding
        const Spacer(),
        TextButton(
          onPressed: () => {debugPrint("Undid")}, child: Text("X")) // TextButton
      ],
    )), // Row // Container
  ); // SnackBar
  ScaffoldMessenger.of(context).showSnackBar(snackBar);
}

showAlertDialog(BuildContext context) {}
}

```

Lalu buat script alert.dart di widgets untuk menampilkan apakah registernya berhasil atau tidak seperti di bawah ini

- showAlert untuk menampilkan SnackBar dengan menunjukkan nontifikasi status berhasil/gagal
- ShowAlertDialog untuk menampilkan dialog konfirmasi dengan tombol cancel dan continue

```

class UserService {
  Future registerUser(data) async {
    var uri = Uri.parse(url.baseUrl + "auth/register");
    var register = await http.post(uri, body: data);

    if (register.statusCode == 200) {
      var data = json.decode(register.body);
      if(data["status"] == true) {
        ResponseDataMap response = ResponseDataMap(
          status: true, message: "Sukses menambah user", data: data);
        return response;
      } else {
        var message = '';
        for (String key in data["message"].keys) {
          message += data["message"][key][0].toString() + '\n';
        }
        ResponseDataMap response =
          ResponseDataMap(status: false, message: message);
        return response;
      }
    } else {
      ResponseDataMap response = ResponseDataMap(
        status: false,
        message:
          "gagal menambah user dengan code error ${register.statusCode}");
      return response;
    }
  }
}

```

```

Future loginUser(data) async {
  var uri = Uri.parse(url.baseUrl + "/auth/login");
  var register = await http.post(uri, body: data);

  if (register.statusCode == 200) {
    var data = json.decode(register.body);
    if (data["status"] == true) {
      > UserLogin userLogin = UserLogin(...
      await userLogin.prefs();
      ResponseDataMap response = ResponseDataMap(
        status: true, message: "Sukses login user", data: data);
      return response;
    } else {
      ResponseDataMap response =
        ResponseDataMap(status: false, message: 'Email dan password salah');
      return response;
    }
  } else {
    ResponseDataMap response = ResponseDataMap(
      status: false,
      message:
        "gagal menambah user dengan code error ${register.statusCode}");
    return response;
  }
}

```

Kemudian buat lah file user.dart di folder service seperti script diatas ini

File ini mengirimkan data ke pendaftaran user di API, jika respon suksse di postman akan dikonversi ke DataMap, kalau gagal akan muncul pesan error

```

class _RegisterUserViewState extends State<RegisterUserView> {
  UserService user = UserService();

  final formKey = GlobalKey<FormState>();
  TextEditingController name = TextEditingController();
  TextEditingController email = TextEditingController();
  TextEditingController password = TextEditingController();
  TextEditingController address = TextEditingController();
  TextEditingController phone = TextEditingController();
  List roleChoice = ["casier", "user"];
  String? role;
}

```

Lalu di halaman register user buat menjadi statefull dan juga tambahkan script seperti di atas

- fromKey untuk menyimpan validasi form
- TextEditingController untuk mengontrol input user
- RoleChoise untuk menentukan peran user(cashier/user)

```

DropDownButtonFormField(
  isExpanded: true,
  value: role,
  items: roleChoice.map((r) {
    return DropdownMenuItem(value: r, child: Text(r));
  }).toList(),
  onChanged: (value) {
    setState(() {
      role = value.toString();
    });
  },
  hint: Text("Pilih role"),
  validator: (value) {
    if (value == null) return 'Role harus dipilih';
    return null;
  },
), // DropDownButtonFormField

```

Script di atas digunakan untuk membuat DropDownButton dan memilih role (cashier/user)

```

Future loginUser(data) async {
  var uri = Uri.parse(url.baseUrl + "/auth/login");
  var register = await http.post(uri, body: data);

  if (register.statusCode == 200) {
    var data = json.decode(register.body);
    if (data["status"] == true) {
      > UserLogin userLogin = UserLogin( ...
        await userLogin.prefs();
        ResponseDataMap response = ResponseDataMap(
          status: true, message: "Sukses login user", data: data);
        return response;
      } else {
        ResponseDataMap response =
          ResponseDataMap(status: false, message: 'Email dan password salah');
        return response;
      }
    } else {
      ResponseDataMap response = ResponseDataMap(
        status: false,
        message:
          "gagal menambah user dengan code error ${register.statusCode}");
      return response;
    }
  }
}

```

Script di atas digunakan untuk memvalidasi form dan mengirim data ke API

Jika suksse maka menghapus input, menampilkan pesan sukses, dan pindah ke halaman login dan kalau gagal maka menampilkan pesan error

```

        SizedBox(height: 20),
        TextFormField(
          controller: email,
          decoration: InputDecoration(
            labelText: "Email",
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(10),
            ), // OutlineInputBorder
          ), // InputDecoration
          validator: (value) {
            if (value!.isEmpty) {
              return 'Email harus diisi';
            }
            return null;
          },
        ), // TextFormField
        SizedBox(height: 15),
        TextFormField(
          controller: password,
          obscureText: showPass,
          decoration: InputDecoration(
            labelText: "Password",
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(10),
            ), // OutlineInputBorder
            suffixIcon: IconButton(
              icon: Icon(
                showPass ? Icons.visibility : Icons.visibility_off,
              ), // Icon
              onPressed: () {
                setState(() {
                  showPass = !showPass;
                });
              },
            ), // IconButton
          ), // InputDecoration
          validator: (value) {
            if (value!.isEmpty) {
              return 'Password harus diisi';
            }
            return null;
          },
        ), // TextFormField
        SizedBox(height: 10),
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            Row(
              children: [
                Checkbox(
                  value: rememberMe,
                  onChanged: (value) {
                    setState(() {
                      rememberMe = value!;
                    });
                  },
                ), // Checkbox
                Text("Remember me")
              ],
            ), // Row
            TextButton(
              onPressed: () {},
              child: Text("Forgot password?",
                style:
                  TextStyle(color: Color(0xFF3B5E9F)), // Text
              ), // TextButton
            ),
          ],
        ), // Row

```

Setelah itu, tambahkan fungsi ini untuk membuat input form dengan ikon dna validasi

Di bawah ini adalah kode lengkap beserta desain nya

```
import 'package:flutter/material.dart';
import 'package:toko_online/services/user.dart';
import 'package:toko_online/widgets/alert.dart';

class LoginView extends StatefulWidget {
  const LoginView({super.key});

  @override
  State<LoginView> createState() => _LoginViewState();
}

class _LoginViewState extends State<LoginView> {
  UserService user = UserService();
  final formKey = GlobalKey<FormState>();
  TextEditingController email = TextEditingController();
  TextEditingController password = TextEditingController();
  bool isLoading = false;
  bool showPass = true;
  bool rememberMe = false;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SingleChildScrollView(
        child: Column(
          children: [
            Stack(
              children: [
                Container(
                  height: 250,
                  width: double.infinity,
                  decoration: BoxDecoration(
                    gradient: LinearGradient(
                      colors: [Color(0xFF3B5E9F), Color(0xFF6C8CFF)],
                      begin: Alignment.topLeft,
                      end: Alignment.bottomRight,
                    ), // LinearGradient
                    borderRadius: BorderRadius.vertical(
                      bottom: Radius.circular(50),
                    ), // BorderRadius.vertical
                  ), // BoxDecoration
                ), // Container
                SafeArea(
                  child: Padding(
                    padding: const EdgeInsets.all(16.0),
                    child: GestureDetector(
                      onTap: () => Navigator.pop(context),
                      child: CircleAvatar(
                        backgroundColor: Colors.black.withOpacity(0.3),
                        child: Icon(Icons.arrow_back, color: Colors.white),
                      ), // CircleAvatar
                    ), // GestureDetector
                  ), // Padding
                ), // SafeArea
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

```

), // Stack
Container(
  transform: Matrix4.translationValues(0.0, -40.0, 0.0),
  margin: EdgeInsets.symmetric(horizontal: 20),
  padding: EdgeInsets.all(20),
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(20),
    boxShadow: [
      BoxShadow(
        color: Colors.grey.withOpacity(0.3),
        spreadRadius: 2,
        blurRadius: 7,
        offset: Offset(0, 3),
      ), // BoxShadow
    ],
  ), // BoxDecoration
  child: Form(
    key: formKey,
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Center(
          child: Text(
            "Welcome back",
            style: TextStyle(
              fontSize: 22,
              fontWeight: FontWeight.bold,
              color: Color(0xFF3B5E9F),
            ), // TextStyle
          ), // Text
        ), // Center
        SizedBox(height: 20),
        TextFormField(
          controller: email,
          decoration: InputDecoration(
            labelText: "Email",
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(10),
            ), // OutlineInputBorder
          ), // InputDecoration
          validator: (value) {
            if (value!.isEmpty) {
              return 'Email harus diisi';
            }
            return null;
          },
        ), // TextFormField
        SizedBox(height: 15),
        TextFormField(
          controller: password,
          obscureText: showPass,
          decoration: InputDecoration(

```



```
SizedBox(height: 10),
SizedBox(
  width: double.infinity,
  child: ElevatedButton(
    onPressed: () async {
      if (formKey.currentState!.validate()) {
        setState(() {
          isLoading = true;
        });
        var data = {
          "email": email.text,
          "password": password.text,
        };
        var result = await user.loginUser(data);
        setState(() {
          isLoading = false;
        });
        if (result.status == true) {
          AlertMessage()
            .showAlert(context, result.message, true);
          Future.delayed(Duration(seconds: 2), () {
            Navigator.pushReplacementNamed(
              context, '/dashboard');
          }); // Future.delayed
        } else {
          AlertMessage()
            .showAlert(context, result.message, false);
        }
      }
    },
    style: ElevatedButton.styleFrom(
      backgroundColor: Color(0xFF385EFF),
      padding: EdgeInsets.symmetric(vertical: 14),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(10),
      ), // RoundedRectangleBorder
    ),
    child: isLoading
      ? CircularProgressIndicator(color: Colors.white)
      : Text("Sign in"),
  ), // ElevatedButton
), // SizedBox
SizedBox(height: 20),
Center(child: Text("Sign in with")),
SizedBox(height: 10),
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Icon(Icons.facebook, color: Colors.blue),
    SizedBox(width: 10),
    Icon(Icons.g_mobiledata, color: Colors.red),
    SizedBox(width: 10),
    Icon(Icons.apple, color: Colors.black),
  ],
)
```

```

        SizedBox(height: 20),
        Center(child: Text("Sign in with")),
        SizedBox(height: 10),
        Row(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Icon(Icons.facebook, color: Colors.blue),
            SizedBox(width: 10),
            Icon(Icons.g_mobiledata, color: Colors.red),
            SizedBox(width: 10),
            Icon(Icons.apple, color: Colors.black),
          ],
        ), // Row
        SizedBox(height: 15),
        Center(
          child: GestureDetector(
            onTap: () {
              Navigator.pushNamed(context, '/register-user');
            },
            child: RichText(
              text: TextSpan(
                style: TextStyle(color: Colors.black),
                children: [
                  TextSpan(text: "Don't have an account? "),
                  TextSpan(
                    text: "Sign up",
                    style: TextStyle(color: Color(0xFF3B5E9F)),
                  ), // TextSpan
                ],
              ), // TextSpan
            ), // RichText
          ), // GestureDetector
        ), // Center
      ],
    ), // Column
  ), // Form
), // Container
],
), // Column
), // SingleChildScrollView
); // Scaffold
}
}

```